

Mini-Store-ERP — Project Documentation Guide

1. Introduction

The **Mini-Store-ERP** is a full-stack MERN-based (MongoDB, Express.js, React.js, Node.js) Enterprise Resource Planning solution designed for small and medium-sized retail stores. This documentation guide provides a comprehensive overview of the system architecture, design decisions, features, technology stack, API structure, and deployment workflow. It serves as a reference manual for developers, testers, and evaluators.

2. Project Overview

2.1 Objective

To build a modern, AI-powered inventory and sales management system that automates daily store operations, provides actionable insights, and ensures accurate stock tracking.

2.2 Key Features

- Secure user authentication with roles (Admin/Cashier)
 - Product catalog & stock management
 - Purchase & supplier tracking
 - POS & sales management with automatic stock deduction
 - PDF invoice generation
 - AI-powered insights using Groq LLM
 - Real-time dashboards & analytics
 - Error handling, validation, and system logging
-

3. System Architecture

The Mini-Store-ERP follows a **3-tier architecture**:

3.1 Frontend

- React + TypeScript
- TailwindCSS for UI
- React Router v6
- Zustand for state management
- Axios for API communication

3.2 Backend

- Node.js + Express.js
- TypeScript-based service layer
- JWT authentication
- Groq API for AI insights
- jsPDF for invoice generation
- Mongoose ORM

3.3 Database

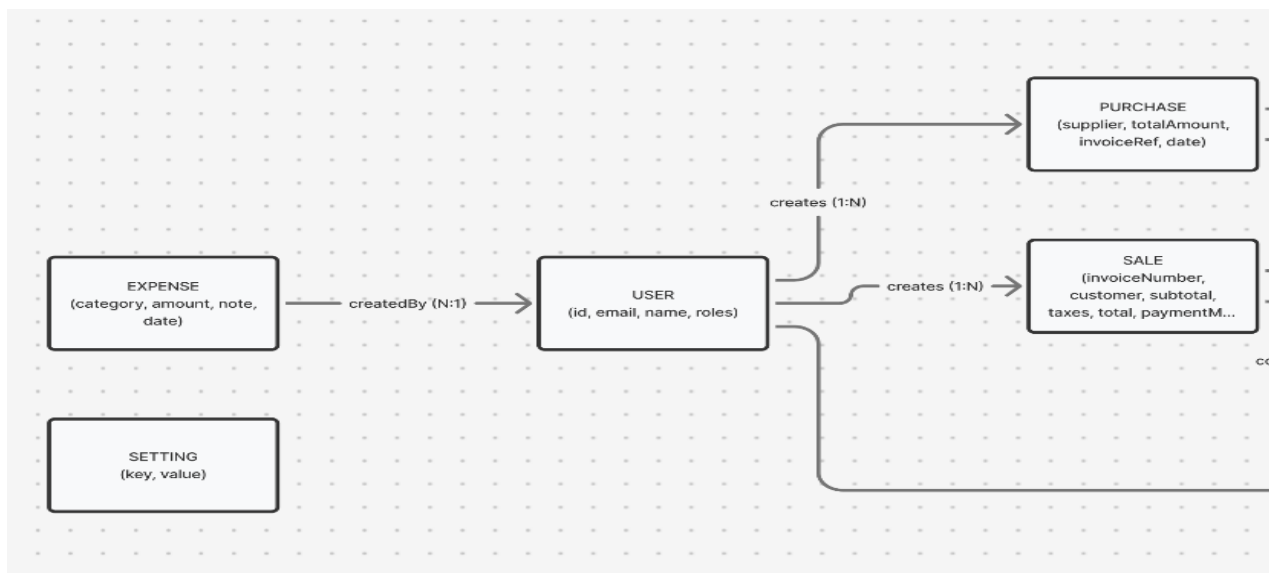
- MongoDB (NoSQL document-oriented database)

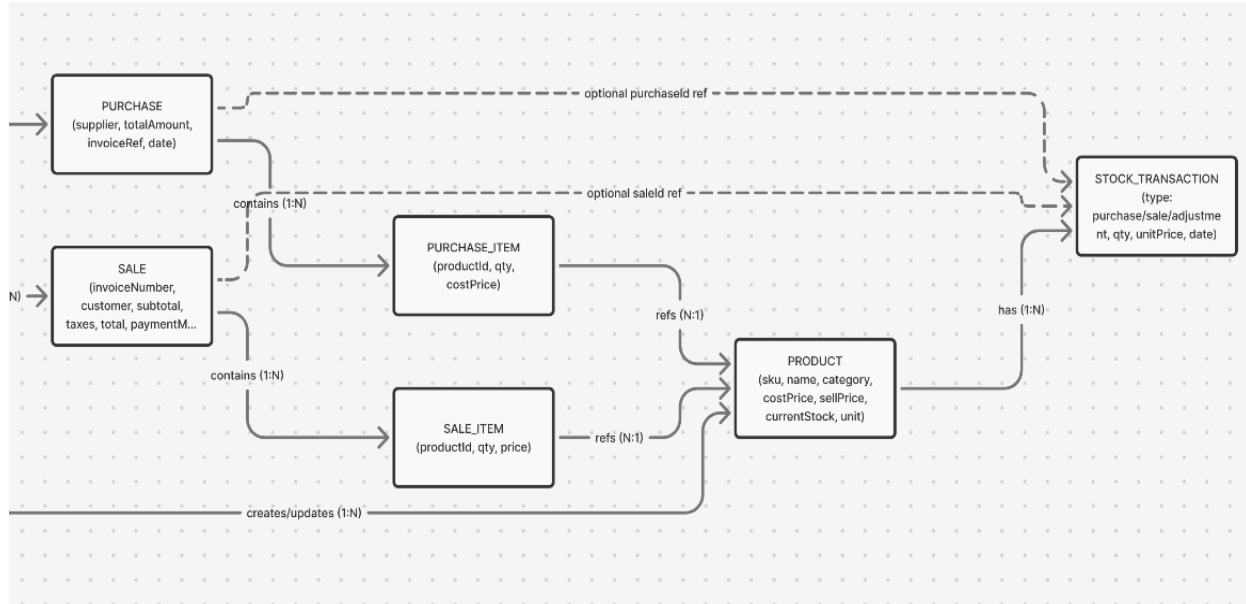
4. ER Diagram & Data Models

Below is the ER diagram summary (as provided):

- **User** ↔ Product (createdBy, updatedBy)
- **Product** ↔ StockTransaction
- **Sale** ↔ StockTransaction
- **Purchase** ↔ StockTransaction
- **Product** ↔ Sale.items[]
- **Product** ↔ Purchase.items[]
- **Expense** linked to User
- **Settings** standalone for system configuration

Full ER diagram is included in the project assets.





5. Backend API Modules

5.1 Authentication API

- Login feature
- Register feature
- Verify token

5.2 Product API

- Create product
- Update product
- Delete product
- Fetch product list

5.3 Purchase API

- Create purchase
- List purchases
- Generate stock additions

5.4 Sales API

- Create sale
- Handle POS checkout
- Auto-generate invoice PDFs

5.5 Stock Transactions API

- Track stock movements
- Adjustment logs

5.6 Expense API

- Add expenses
- Categorize expenses

5.7 AI Insights API

- Natural-language query → JSON insights
 - Smart reorder suggestions
-

6. Frontend and Backend Structure

Key Screens of the Project

- Login/Registration Screen Page
- Dashboard (Revenue, COGS, Top Products)
- Products Page
- Add/Edit Product Page
- Sales POS Page
- Purchases & Suppliers Page
- Reports & Insights Page

Frontend (React + TypeScript) — Complete Structure

```
client/src
├── components/      # Reusable UI components (buttons, modals, tables)
├── pages/           # Main route pages (Dashboard, Products, POS, etc.)
├── hooks/           # Custom hooks (useAuth, useFetch, useStore)
├── context/         # Global context providers
├── layouts/         # Layout components (Sidebar, Navbar,
ProtectedLayout)
├── assets/          # Images, icons, static files
├── utils/           # Helper functions (formatters, validators)
├── store/           # Zustand store for state management
├── lib/             # API handlers, axios wrapper
├── types/           # TypeScript types
├── router/          # Route configuration
├── styles/          # Global styles or theme config
└── config/          # Environment configs, constants
```

Backend (Node.js + Express + TypeScript) — Complete Structure

```
server/
├── src/
│   ├── config/           # DB config, environment loader
│   ├── controllers/      # API controllers for each module
│   ├── middleware/       # Auth, error handling, validation
│   ├── models/           # Mongoose schemas & models
│   ├── services/         # Business logic layer (ProductService, SaleService)
│   ├── routes/           # API route definitions
│   ├── utils/            # Helpers: tokens, logger, PDF tools
│   ├── constants/        # App constants, enums, role permissions
│   ├── validators/       # Request validation schemas
│   └── app.ts            # Express app initialization
├── logs/                 # Winston or custom logs
├── public/               # Public assets (invoice templates)
└── package.json          # Dependencies
```

These structures represent all important directories required for scalable frontend and backend development.

7. AI Integration (Groq LLM)

Use Cases of this Project

1. Natural Language Query Understanding

Users can type queries in plain English, such as:

- “Show me today’s revenue.”
- “Which products are running low?”
- “Top-selling categories this month?”

The backend converts these into structured prompts for the Groq model.

✓ **No need for SQL or filters**

✓ **Fast real-time responses**

2. Business Insights Generation

Groq processes store data and provides insights such as:

- Revenue trends
- Profit vs expense summary
- Top-selling products
- Low-performing items

- Day-wise, week-wise, month-wise metrics
- Supplier performance
- High-cost vs high-profit products

The model returns a **clean JSON** format to be easily displayed in React charts.

3. Smart Reorder Suggestions

This feature calculates:

- Daily sales movement
- Remaining stock
- Lead time (optional future upgrade)
- Reorder thresholds

The AI suggests following:

- Which items to reorder
- Estimated stock-out days
- Recommended reorder quantities

Useful for inventory optimization.

4. Conversational Analytics

The system behaves like a **business analyst chatbot**:

- Answers follow-up queries
- Remembers the context of the previous question (if implemented)
- Delivers short, precise insights

This improves UX over static dashboards.

5. Backend Prompt Engineering

The backend formats the prompt with:

- Sales and Purchase data
- Stock and Expenses data
- Revenue summaries and Date ranges
- Product names, categories, vendors

Before sending to Groq, it ensures:

- No leakage of raw DB schema
- Model-friendly structure

Groq returns a **stable JSON output**, not plain text.

8. PDF Generation

Invoice PDF

- Invoice number
- Customer details
- Itemized list
- Tax + discount
- Total amount

Tech Used

- jsPDF
 - AutoTable plugin
-

9. Deployment Guide

Backend Deployment (Render)

- Build: `cd server; npm install; tsc`
- Start: `cd server; npm start`
- Add environment variables

Frontend Deployment (Render)

- Build: `cd client; npm run build`
 - Publish folder: `client/dist`
 - Add `VITE_API_URL` env
-

10. Environment Variables

Server .env

```
MONGODB_URI="enter your mongodb url here"
JWT_SECRET="enter your jwt secret here"
GROQ_API_KEY="enter your groq api key here"
GROQ_MODEL="enter the model name"
CORS_ORIGIN="enter backend deployment url(production)"
NODE_ENV="development"
JWT_EXPIRE=24h
JWT_REFRESH_EXPIRE=7d
```

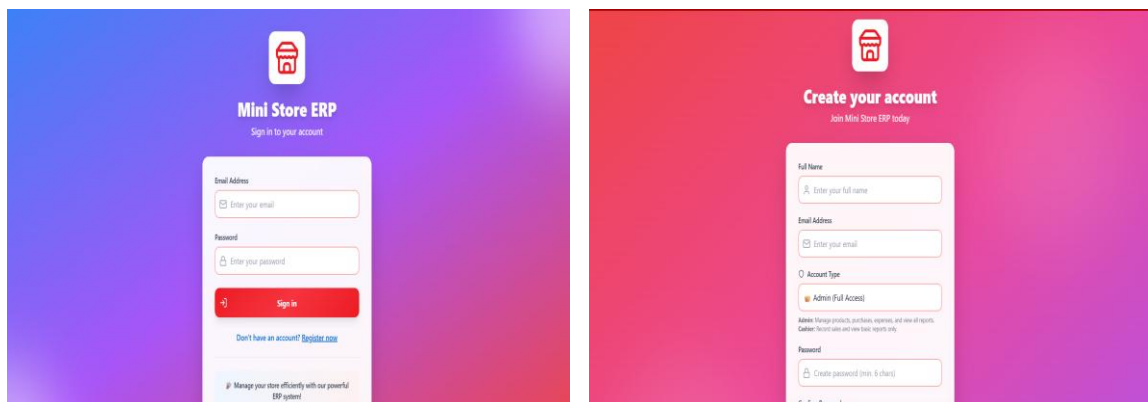
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX_REQUESTS=100

11. Future Enhancements

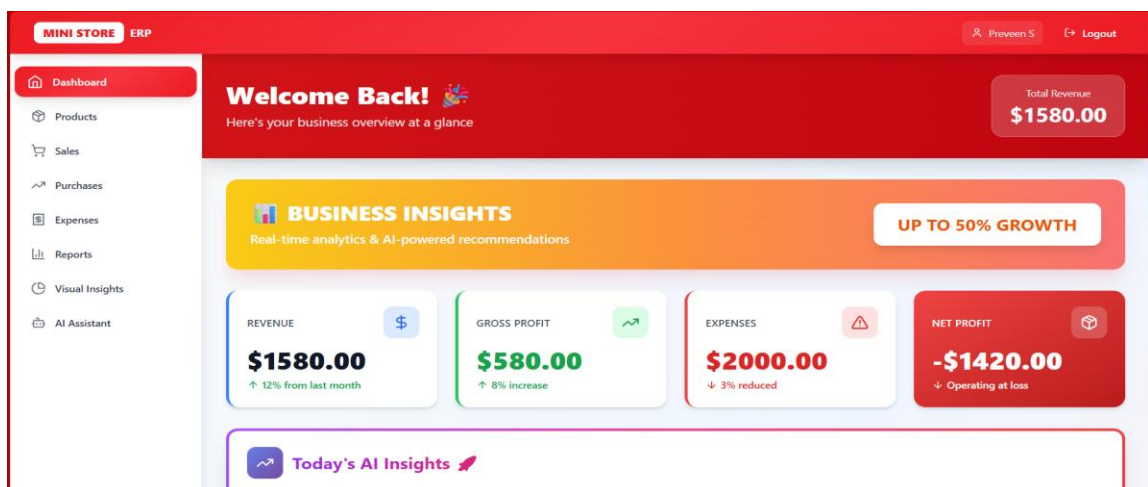
- Multi-branch store support
 - Barcode scanner support
 - Mobile app version
 - Staff attendance & payroll
-

12. Project Screenshots

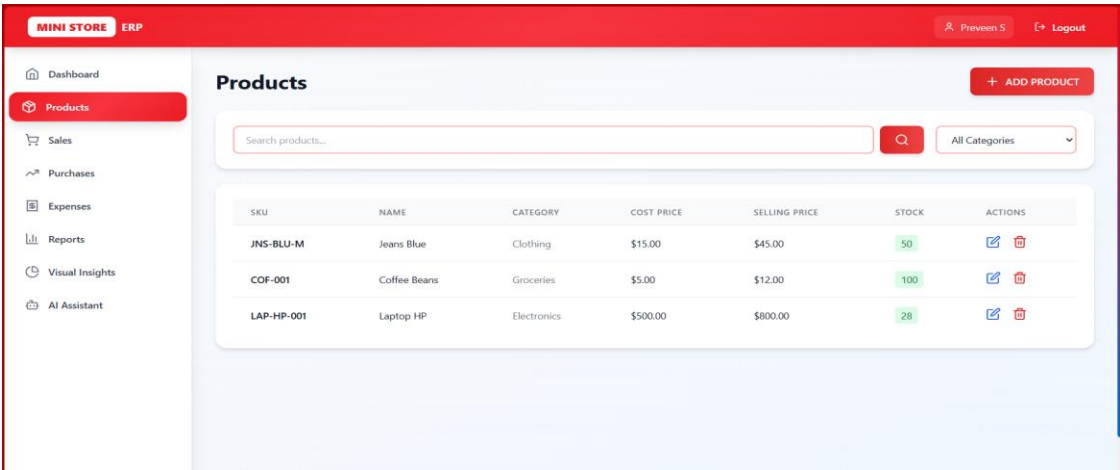
- Login and Register Page



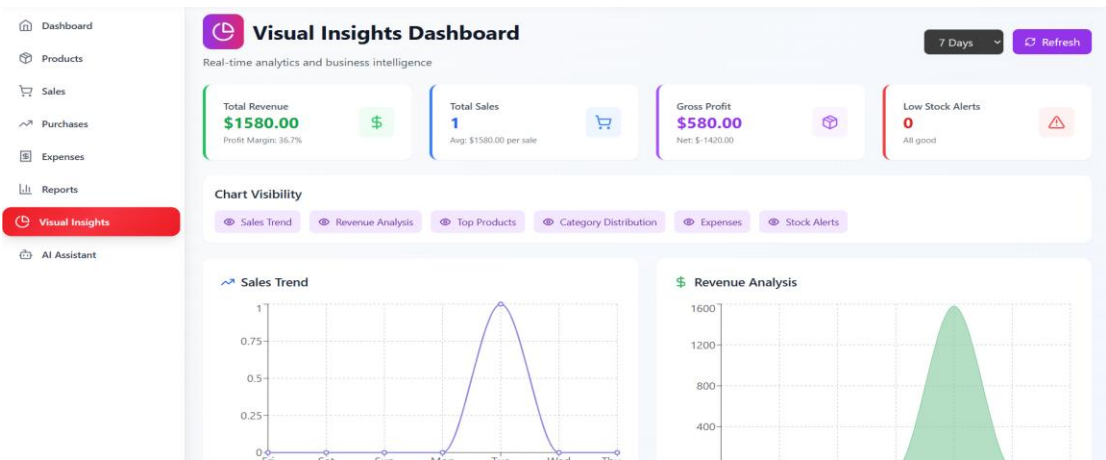
- Dashboard Page



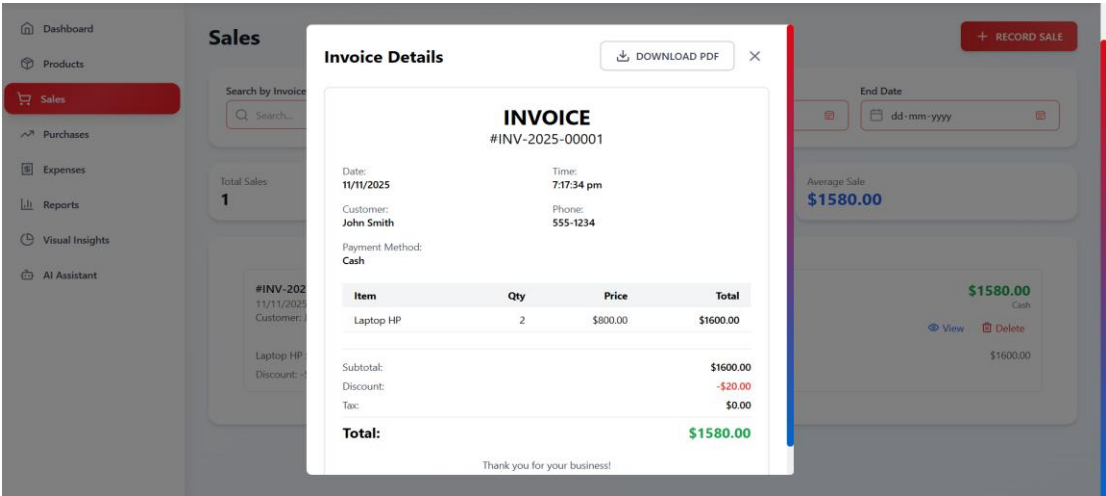
- **Products Page**



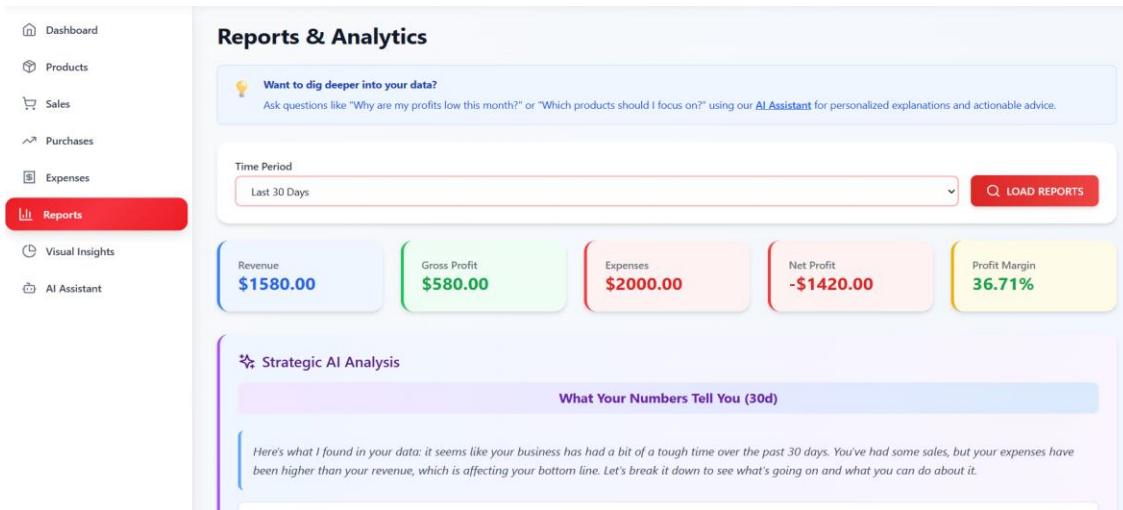
- **Visual Insights Page**



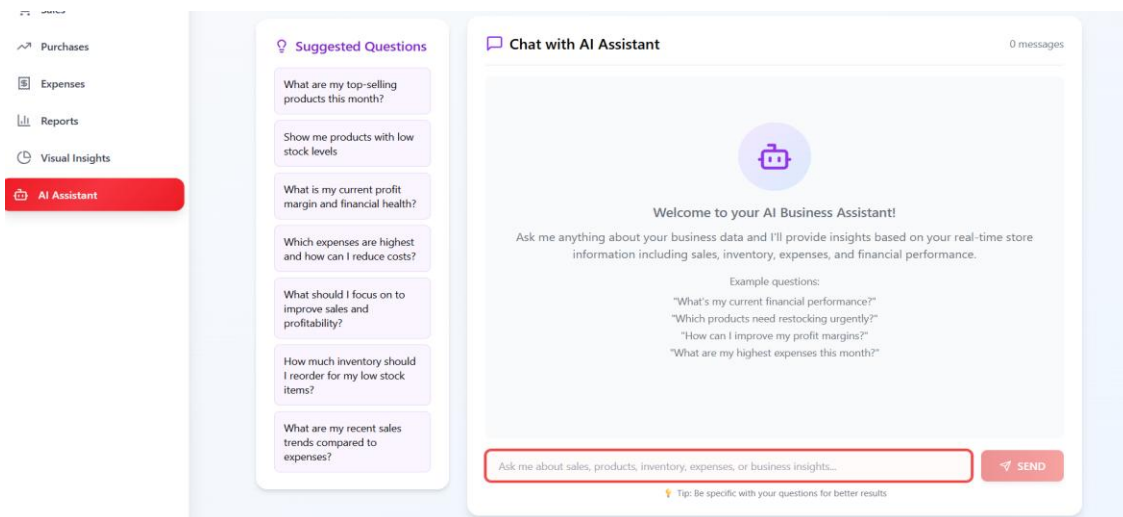
- **Sales Invoice Page**



- **Reports Page**



- **AI Assistant Page**



13. Conclusion

This documentation guide provides the complete overview needed for understanding, deploying, evaluating, and extending the Mini-Store-ERP application. For additional details, refer to README files and source code comments.